JavaScript Array Iterations

Previous Next

Array Iteration Methods

Array iteration methods operate on every array item.

- Array forEach
- Array map()
- Array flatMap()
- Array filter()
- Array reduce()
- Array reduceRight()
- Array every()
- Array some()
- Array from()
- Array keys()
- Array entries()
- Array with()
- Array Spread (...)
- Array Rest (...)

See Also:

Array Tutorial

Array Basic Methods

Array Search Methods

Array Sort Methods

Array Iteration Methods

Array Reference

Example

```
const numbers = [45, 4, 9, 16, 25];
let txt = "";
numbers.forEach(myFunction);

function myFunction(value, index, array) {
  txt += value + "<br>";
}
```

Try it Yourself »

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

The example above uses only the value parameter. The example can be rewritten to:

Example

```
const numbers = [45, 4, 9, 16, 25];
let txt = "";
numbers.forEach(myFunction);

function myFunction(value) {
  txt += value + "<br>";
}
```

Try it Yourself »

JavaScript Array map()

The `map()` method creates a new array by performing a function on each array element.

The `map()` method does not execute the function for array elements without values.

Example

```
const numbers1 = [45, 4, 9, 16, 25];
const numbers2 = numbers1.map(myFunction);

function myFunction(value, index, array) {
  return value * 2;
}
```

Try it Yourself »

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

When a callback function uses only the value parameter, the index and array parameters can be omitted:

Example

```
const numbers1 = [45, 4, 9, 16, 25];
const numbers2 = numbers1.map(myFunction);

function myFunction(value) {
  return value * 2;
}
```

Try it Yourself »

JavaScript Array flatMap()

ES2019 added the Array `flatMap()` method to JavaScript.

The `flatMap()` method first maps all elements of an array and then creates a new array by flattening the array.

```
const myArr = [1, 2, 3, 4, 5, 6];
const newArr = myArr.flatMap((x) => x * 2);
```

Try it Yourself »

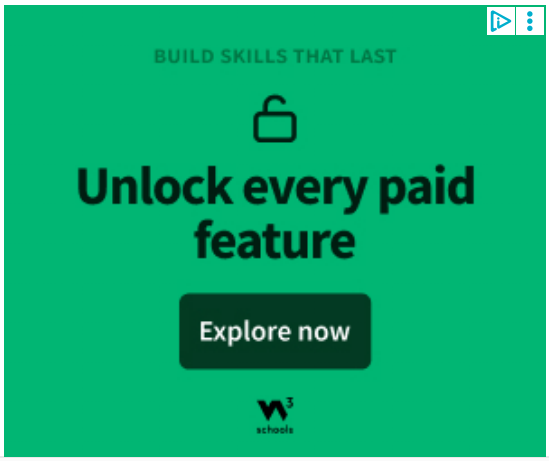
Browser Support

`flatMap()` is an ECMAScript 2019 feature.

ES2019 is supported in all modern browsers since January 2020:

Chrome	Edge	Firefox	Safari	Opera
66	79	61	12	50
Apr 2018	Jan 2020	Jun 2018	Sep 2018	May 2018

ADVERTISEMENT

ADVERTISEMENT


A

JavaScript Array filter()

The `filter()` method creates a new array with array elements that pass a test.

This example creates a new array from elements with a value larger than 18:

Example

```
  return value > 18;
}
```

Try it Yourself »

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

In the example above, the callback function does not use the index and array parameters, so they can be omitted:

Example

```
const numbers = [45, 4, 9, 16, 25];
const over18 = numbers.filter(myFunction);

function myFunction(value) {
  return value > 18;
}
```

Try it Yourself »

JavaScript Array reduce()

The `reduce()` method runs a function on each array element to produce a single value.

The `reduce()` method works from left-to-right in the array. See also `reduceRight()`.

Note

The `reduce()` method does not reduce the original array.

This example finds the sum of all numbers in an array:

```
let sum = numbers.reduce(myFunction);

function myFunction(total, value, index, array) {
  return total + value;
}
```

Try it Yourself »

Note that the function takes 4 arguments:

- The total (the initial value / previously returned value)
- The item value
- The item index
- The array itself

Since the example above does not use the index and array parameters, it can be rewritten to:

Example

```
const numbers = [45, 4, 9, 16, 25];
let sum = numbers.reduce(myFunction);

function myFunction(total, value) {
  return total + value;
}
```

Try it Yourself »

The `reduce()` method can accept an initial value:

Example

```
const numbers = [45, 4, 9, 16, 25];
let sum = numbers.reduce(myFunction, 100);

function myFunction(total, value) {
  return total + value;
}
```

JavaScript Array reduceRight()

The `reduceRight()` method runs a function on each array element to produce a single value.

The `reduceRight()` works from right-to-left in the array. See also `reduce()`.

Note

The `reduceRight()` method does not reduce the original array.

This example finds the sum of all numbers in an array:

Example

```
const numbers = [45, 4, 9, 16, 25];
let sum = numbers.reduceRight(myFunction);

function myFunction(total, value, index, array) {
  return total + value;
}
```

Try it Yourself »

Note that the function takes 4 arguments:

- The total (the initial value / previously returned value)
- The item value
- The item index
- The array itself

The example above does not use the index and array parameters. It can be rewritten to:

Example

```
const numbers = [45, 4, 9, 16, 25];
let sum = numbers.reduceRight(myFunction);
```

Try it Yourself »

JavaScript Array every()

The `every()` method checks if all array values pass a test.

This example checks if all array values are larger than 18:

Example

```
const numbers = [45, 4, 9, 16, 25];
let allOver18 = numbers.every(myFunction);

function myFunction(value, index, array) {
  return value > 18;
}
```

Try it Yourself »

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

When a callback function uses the first parameter only (value), the other parameters can be omitted:

Example

```
const numbers = [45, 4, 9, 16, 25];
let allOver18 = numbers.every(myFunction);

function myFunction(value) {
  return value > 18;
}
```

Try it Yourself »

The `some()` method checks if some array values pass a test.

This example checks if some array values are larger than 18:

Example

```
const numbers = [45, 4, 9, 16, 25];
let someOver18 = numbers.some(myFunction);

function myFunction(value, index, array) {
  return value > 18;
}
```

Try it Yourself »

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

JavaScript Array.from()

The `Array.from()` method returns an Array object from:

- **Any iterable object**
- **Any object with a length property**

Example

Create an Array from a String:

```
let text = "ABCDEFG";
Array.from(text);
```

Try it Yourself »

Example

Create an Array from an Array:

```
const myNumbers = [1,2,3,4];
const myArr = Array.from(myNumbers, (x) => x * 2);
```

Try it Yourself »

Browser Support

`from()` is an ES6 feature.

ES6 is fully supported in all modern browsers since June 2017:

Chrome	Edge	Firefox	Safari	Opera
51	15	54	10	38
May 2016	Apr 2017	Jun 2017	Sep 2016	Jun 2016

JavaScript Array keys()

The `Array.keys()` method returns an Array Iterator object with the keys of an array.

Example

Create an Array Iterator object, containing the keys of the array:

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
const keys = fruits.keys();

for (let x of keys) {
  text += x + "<br>";
}
```

Try it Yourself »

ES6 is fully supported in all modern browsers since June 2017:

Chrome 51	Edge 15	Firefox 54	Safari 10	Opera 38
May 2016	Apr 2017	Jun 2017	Sep 2016	Jun 2016

JavaScript Array entries()

Example

Create an Array Iterator, and then iterate over the key/value pairs:

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
const f = fruits.entries();

for (let x of f) {
  document.getElementById("demo").innerHTML += x;
}
```

Try it Yourself »

The `entries()` method returns an Array Iterator object with key/value pairs:

[0, "Banana"]
[1, "Orange"]
[2, "Apple"]
[3, "Mango"]

The `entries()` method does not change the original array.

Browser Support

`entries()` is an ES6 feature.

ES6 is fully supported in all modern browsers since June 2017:

May 2016	Apr 2017	Jun 2017	Sep 2016	Jun 2016

JavaScript Array with() Method

ES2023 added the Array with() method as a safe way to update elements in an array without altering the original array.

Example

```
const months = ["Januar", "Februar", "Mar", "April"];
const myMonths = months.with(2, "March");
```

Try it Yourself »

JavaScript Array Spread (...)

The `...` operator expands an array into individual elements.

This can be used join arrays:

Example 1

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];

const arr3 = [...arr1, ...arr2];
```

Try it Yourself »

In the example above, **...arr1** expands arr1 into single elements, **...arr2** expands arr2 into single elements, and arr3 is constructed using ...arr1 and ...arr2.

Example 2

```
const year = [...q1, ...q2, ...q3, ...q4];
```

Try it Yourself »

The spread operator (...) can be used to copy an array:

Example 3

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1];
```

Try it Yourself »

The spread operator (...) can be used to pass arguments to a function:

Example 4

```
const numbers = [23,55,21,87,56];
let minValue = Math.min(...numbers);
let maxValue = Math.max(...numbers);
```

Try it Yourself »

Browser Support

`... (spread)` is an ES6 feature.

ES6 is fully supported in all modern browsers since June 2017:

Chrome	Edge	Firefox	Safari	Opera
51	15	54	10	38
May 2016	Apr 2017	Jun 2017	Sep 2016	Jun 2016

Example 1

```
let a, rest;
const arr1 = [1,2,3,4,5,6,7,8];

[a, ...rest] = arr1;
```

Try it Yourself »

Example 2

```
let a, b, rest;
const arr1 = [1,2,3,4,5,6,7,8];

[a, b, ...rest] = arr1;
```

Try it Yourself »

Browser Support

`... (rest)` is an ECMAScript 2018 feature.

ES2018 is supported in all modern browsers since January 2020:

Chrome 64	Edge 79	Firefox 58	Safari 12	Opera 51
Jan 2018	Jan 2020	Jan 2018	Sep 2018	Feb 2018

Complete JavaScript Reference

For a complete reference to all JavaScript properties and methods, with full descriptions and many examples, go to:

Exercise

Consider the following code:

```
const x = [45, 4, 9, 16, 25];
let y = x.every(myfunc);
function myfunc(val) {
  return val > 20;
}
```

What will the result of the `y`?

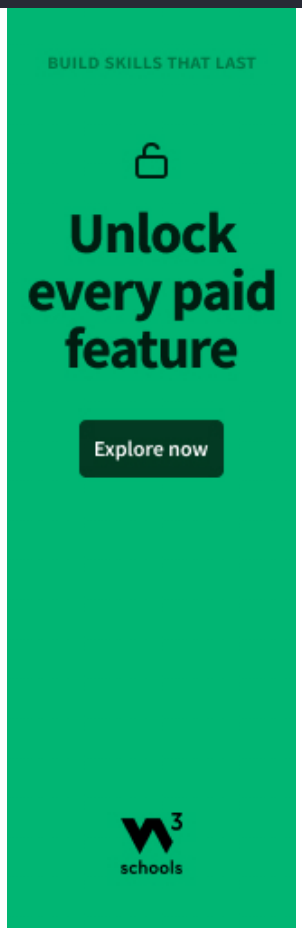
- [45, 25]
- false
- []
- NaN

Submit Answer »

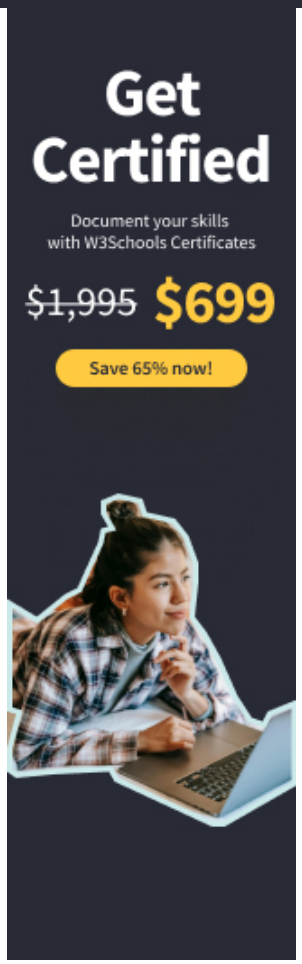
❮ Previous Next ❯

Track your progress - it's free!
Sign Up Log in

ADVERTISEMENT



COLOR PICKER



ADVERTISEMENT

ADVERTISEMENT

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
Angular Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

FORUM ABOUT ACADEMY

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning. Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookie and privacy policy.

Copyright 1999-2025 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.